

Lab Programs

Program 1

- Create a component called Profile in a file named Profile.js. This component should return name of the person as heading, image of a person and description of the person. Also, export the component as the default export.
- Create another component called Gallery in a file named Gallery.js. This Gallery component should import the Profile component and export itself as a named export.
- Inside App component render Gallery component

Program 2

- Create a JavaScript object named scientist1 that contains details about the first scientist (e.g., id, name, image, profession, and description). Create a second JavaScript object named scientist2 with details about a second scientist
- Create a component called Profile in a file named Profile.js. This Profile component should take the scientist's details as props and render them (e.g., name, image, profession, and description).
- Create a component called Card in a file named Card.js. The Card component should access the Profile component using the children prop.
- Inside the App component, render two Card components. Each Card component should wrap a Profile component, and you should pass the scientist1 object as props to the first Profile component and the scientist2 object as props to the second Profile component

Program 3

- Create a JavaScript object named productsData that contains an array of product objects. Each product object should include details such as id, name, price, description, and available (a boolean indicating whether the product is in stock or not)
- Create a component named ProductList in a file named ProductList.js. The ProductList component should map through the productsData array and render a Product component for each product. Pass the product object as props to each Product component.
- Create a component named Product in a file named Product.js. The Product component should receive the product object as props and conditionally render different content based on the available property. If the product is available (available : true), display its name, price, and description. If the product is not available (available : false), display a message saying "Out of stock".
- Inside the App component, render the ProductList component

Program 4

Develop a Login component utilizing form elements as outlined below:

Username:

Password:

Skswamy

Additionally, implement form validation. If the user attempts to submit without entering a username or password, the system should display an error indicating empty fields. Otherwise, upon successful completion, the login should be acknowledged as successful.

Program 5

Create a registration form like the one shown below and make sure each part of the form has validation

Registration Form

Name:

Email:

Password:

Confirm Password:

Gender:
Male ☐ **Female** ☐ **Other** ☐

Hobbies:
Reading ☐ **Traveling** ☐ **Sports** ☐

Country:

Program 6

Create a student management system using React. Implement routing to manage different views such as a student list and individual student profiles. Develop functionalities within the admin panel for adding, editing, and removing students, ensure seamless navigation between these views while displaying relevant information on each page

Program 7

Develop an Express.js application that efficiently handles routes for various shopping sections such as '/products', '/cart', and '/checkout'. Implement custom middleware in Express.js to log essential information from incoming requests, including request method, URL, timestamp, and client's IP address

Program 8

Create Node.js application with the appropriate MongoDB driver, that handles the following CRUD operations for the employee database:

- Implement an endpoint to add new employee to the employee collection. Each employee should have fields such as employeeID, name and designation.
- Develop an endpoint to retrieve employees from the collection based on various search criteria employeeID.
- Provide an endpoint that allows users to update specific details of an employee (e.g., change name, update designation).
- Create an endpoint to remove employee from the collection based on employeeID.

Course Outcomes:

Upon successful completion of this course, student will be able to

CO1	Design, and style React applications, incorporating components, JSX, and basic project structure for effective user interface development
CO2	Demonstrate proficiency in managing component state, passing and utilizing props, and implementing parent-child component communication within React applications.
CO3	Design and implement dynamic forms, handle form data and events, apply form validation techniques, and configure routing for multi-page React applications.
CO4	Demonstrate proficiency in applying Node.js to set up a web server, utilizing NPM for package management, implementing routing and middleware in Express.js
CO5	Design MongoDB schemas, performing CRUD operations, and effectively connecting and interacting with MongoDB databases in web applications.

Mapping of CO-PO:

CO/PO	PO1	PO2	PO3	PO4	PO5
CO1	2	-	2	1	1
CO2	2	-	2	1	1
CO3	2	-	2	1	1
CO4	2	-	2	1	1
CO5	2	-	2	1	1
Average	2	-	2	1	1

Low-1: Medium-2: High-3

Skrwamy